Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Дисциплина «NoSQL»

Лабораторная работа №1

Key-value хранилища
Вариант: 1868679

Выполнил: Решетников С. Е.
Проверил: Цю Тяньшэн

Санкт-Петербург, 2026

1. Задание

Разработать веб-приложение на языке Java с использованием Spring Boot, реализующее сервис заявок

Предметная область: библиотека.
Основной объект системы: событие.
Пользовательская роль: менеджер.
Обязательный сценарий: оформление заказа.

Система хранения типа «ключ — значение»: Etcd. Дополнительные параметры

- сценарий временного хранения: временная заявка;
- сценарий кэширования: настройки пользователя;
- атомарный механизм: счётчик просмотров объекта;
- исследование поведения системы: конфликт параллельных записей.

Требования к программе

1. Развернуть систему хранения, указанную в варианте.
2. Реализовать подключение к ней из Java/Spring Boot.
3. Выбрать и обосновать способ представления данных в модели «ключ — значение».
4. Реализовать хранение части данных приложения в выбранной системе.
5. Реализовать механизм временных данных: TTL для Redis, application-level expiration для Riak KV или lease для Etcd.
6. Реализовать кэширование часто запрашиваемых данных.
7. Реализовать назначенный атомарный счётчик, ограничение запросов или блокировку.
8. Настроить сохранение и восстановление данных, если выбранная система поддерживает этот механизм.
9. Исследовать назначенный сценарием режим поведения системы.
10. Сделать выводы о применимости выбранного хранилища для разработанного приложения.

2. Исполнение

Исходный код может быть найден в ветке *dev* https://github.com/NF-itmo/NoSQL_lab1

2.1. Основные сценарии

1. **Просмотр каталога** По *GET /api/v1/books* книги извлекаются из Etcd, дополняются состоянием доступности и счётчиком, после чего фильтруются по жанру и доступности

2. **Создание заявки** По *POST /api/v1/orders* проверяется существование и доступность книги, затем заявка сохраняется с lease. В текущей модели *orderId* = *bookId*, поэтому на книгу может существовать только одна активная заявка
3. **Подтверждение выдачи** По *POST /api/v1/orders/{orderId}/confirm* Etcd-транзакция проверяет, что заявка существует, а книга доступна. При успехе книга атомарно помечается выданной, а заявка удаляется

2.2. Хранение данных

Данные разделены между PostgreSQL и Etcd. PostgreSQL хранит долгоживущие данные - учётные записи, постоянные настройки пользователей, и каталог книг

```
CREATE TABLE IF NOT EXISTS Users (
  id SERIAL PRIMARY KEY,
  username VARCHAR(60) UNIQUE NOT NULL,
  password_hash VARCHAR(60) NOT NULL
);

CREATE TABLE IF NOT EXISTS Books (
  id INTEGER PRIMARY KEY,
  title TEXT NOT NULL CHECK (length(trim(title)) > 0),
  author TEXT NOT NULL CHECK (length(trim(author)) > 0),
  genre TEXT NOT NULL CHECK (length(trim(genre)) > 0),
  description TEXT,
  available BOOLEAN NOT NULL
);

CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE INDEX IF NOT EXISTS idx_books_title_fts
  ON Books USING GIN (lower(title) gin_trgm_ops);

CREATE INDEX IF NOT EXISTS idx_books_availabillity
  ON Books (available, id);

CREATE TABLE IF NOT EXISTS User_settings (
  username VARCHAR(60) PRIMARY KEY,
  title_filter VARCHAR(100) NOT NULL
);
```

Etcd хранит статистику просмотров книги, заявки и кэш для фильтра по названию и посисковых запросов

Ключ	Формат	Назначение
`/books/search/{requestParamsHash}`	JSON	Кэш поисковых запросов
`/books/availability/{bookId}`	bool	Текущая доступность книги
`/books/views/{bookId}`	int	Счётчик просмотров

Ключ	Формат	Назначение
`/orders/{orderId}`	JSON	Временная заявка
`/cache/user-settings/title-filter/{username}`	String	Последнее значение фильтра по названию

3. Исследование параллельных записей

Возникает при попытке чтения и последующей модификации несколькими процессами - один из результатов может перетереть другой

3.1. Оптимистическая блокировка для числа просмотров

При чтении значения также запоминается и его версия. При обновлении она сравнивается с текущим значением - если совпало, то всё гуд, если нет - ошибка

Из плюсов - не требует блокировать строку, поэтому оказывает минмимальный эффект на производительность. Минус - если «неудачных» раскладов много, то приводит к многократному повторению одной и той же операции

В нашем случае мы:
1. Читаем значение по ключу
2. В транзакции сравниваем modRevision прочитанного и записываемого сецчас объекта
3. Не совпало - перезапуск

3.1.1. Методика

Исследование реализовано в *EtcdConcurrentWritesIT* . Он сам и необходимый кластер запускаются через

```
make test-etcd-conflicts
```

Потоки доходят до барьера *CountDownLatch* и запускаются одновременно, что повышает вероятность конфликтов

3.1.2. Оптимистическая блокировка

100 задач, выполняемых пулом из 20 потоков, увеличивают счётчик */books/views/42*. Измеряются итоговое значение, число потерянных обновлений и набор значений, возвращённых операциями. Корректный набор должен содержать все числа от 1 до 100

3.1.3. Результаты измерений

```
optimistic-locking: operations=100, workers=20, final-value=100, lost-updates=0
```

3.2. Конфликт параллельных записей при работе с заявками

Он исключён благодаря алгоритму создания ключа для заявки - идентификатор книги сопадает с идентификатором заявки, а значит и конфликт принципиально невозможен

4. Оценка применимости

Честно говоря, в данном проекте *etcd* излишен. Тут скорее бы подошёл Redis, и то, только для кэширования часто запрашиваемых данных